Distributed Ride-Sharing System – Project Plan

1. Overview

Build a distributed ride-sharing system on AWS with microservices. Goal: learn system design, distributed systems, and AWS deployment.

2. Tech Stack

- Backend: Node.js / Express (REST APIs)
- Frontend: React (hosted on S3 + CloudFront)
- Database: PostgreSQL (RDS or EC2)
- Caching: Redis (EC2)
- Messaging: Kafka (local or AWS MSK if budget allows)
- Deployment: Docker + ECS/EC2, CI/CD with GitHub Actions
- Networking: API Gateway/ALB, Route 53 (optional custom domain)
- Security: IAM, Security Groups, HTTPS (ACM)

3. Architecture

1. Clients interact with frontend (React) → hosted on S3 + CloudFront.
2. Frontend calls API Gateway/Load Balancer → routes to microservices.
3. Microservices (Auth, Rider, Driver, Matching, Payment) run in ECS/EC2 containers.
4. PostgreSQL + Redis hosted on one EC2 or RDS for persistence + caching.
5. Kafka (optional local for dev) handles async events (ride requests, driver updates).
6. CI/CD pipelines automate deployments.
7. Monitoring via CloudWatch + logs.

4. Deployment Strategy

- Must be on AWS: APIs, DB, Cache, Frontend.
- Can be local: Kafka (replace with Redis pub/sub for deployed demo).
- Start with single EC2 (t2.micro/t3.micro) and Dockerize multiple services.
- Scale later by splitting into ECS tasks or separate EC2s.

5. Cost Estimation

- EC2 t2.micro/t3.micro: Free tier eligible, ~$9/month if outside.
- RDS: ~$15–20/month (or run Postgres in EC2 for free).
- S3 + CloudFront: <$2/month unless heavy traffic.
- Total: $20–30/month (with free tier, possibly $0–10).

6. Development Flow (Step-by-Step)

1. Design database schema (users, rides, payments).
2. Build core backend services locally (Auth, Rider, Driver, Matching, Payment).
3. Add Redis for caching ride lookups.
4. Add Kafka locally for async events (simulate high load).
5. Create frontend (React) with booking flow.
6. Dockerize services.
7. Deploy minimal setup on AWS (EC2 with Postgres+Redis, APIs in containers).
8. Deploy frontend to S3 + CloudFront.
9. Add CI/CD pipeline with GitHub Actions.
10. Add monitoring + polish (logs, metrics, alerts).

7. Timeline (Rough)

- Week 1: DB schema + Auth service
- Week 2: Rider/Driver + Matching service
- Week 3: Payment service + Redis caching
- Week 4: Kafka integration locally
- Week 5: Frontend (React UI)
- Week 6: Dockerize + Deploy to AWS
- Week 7: CI/CD + Monitoring
- Week 8: Final testing + polish